

Условный оператор if-else

Мы познакомились с базовыми строительными блоками программ, научились писать программы, обеспечивающие ввод, обработку и вывод данных. Более того, умеем работать со строками и числами, как мы делаем это в математике. Теперь научимся управлять ходом выполнения программы.

Программы должны уметь выполнять разные действия в зависимости от введенных данных. Для принятия решения программа проверяет, истинно или ложно определенное условие.

В Python существует несколько способов проверки, и в каждом случае возможны два исхода: истина (True) или ложь (False).

Проверка условий и принятие решений по результатам этой проверки называется **ветвлением** (branching). Программа таким способом выбирает, по какой из возможных ветвей ей двигаться дальше.

В Python проверка условия осуществляется при помощи ключевого слова `if`.

Рассмотрим следующую программу:

```
answer = input('Какой язык программирования мы изучаем?')

if answer == 'Python':

    print('Верно! Мы ботаем Python =)')

    print('Python - отличный язык!')
```

Программа просит пользователя ввести текст и проверяет результат ввода. Если введенный текст равен строке «Python», то выводит пользователю текст:

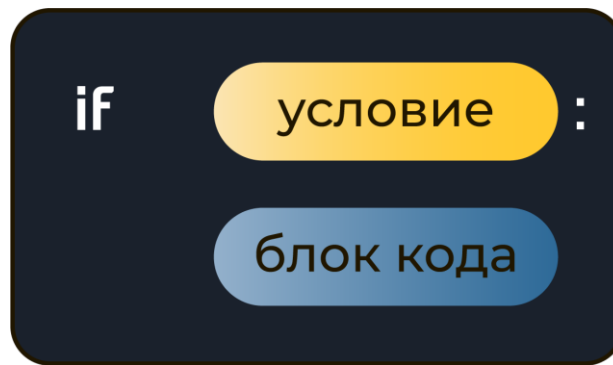
```
Верно! Мы ботаем Python =)

Python - отличный язык!
```

Двоеточие (:) в конце строки с инструкцией `if` сообщает интерпретатору Python, что дальше находится **блок команд**. В блок команд входят все строки с отступом под строкой с инструкцией `if`, вплоть до следующей строки без отступа.

Если условие истинно, выполняется весь расположенный ниже блок. В предыдущем примере блок инструкций составляет третья и четвёртая строки программы.

Блоком кода называют объединённые друг с другом строки. Они всегда связаны с определённой частью программы (например, с инструкцией `if`). В Python блоки кода формируются при помощи **отступов**.



Предыдущая программа выводит текст в случае, если условие истинно. Но если условие ложно, то программа ничего не выводит. Для того, чтобы обеспечить возможность выполнять что-либо, если условие оказалось ложным, мы используем ключевое слово `else`.

```
answer = input('Какой язык программирования мы изучаем?')

if answer == 'Python':

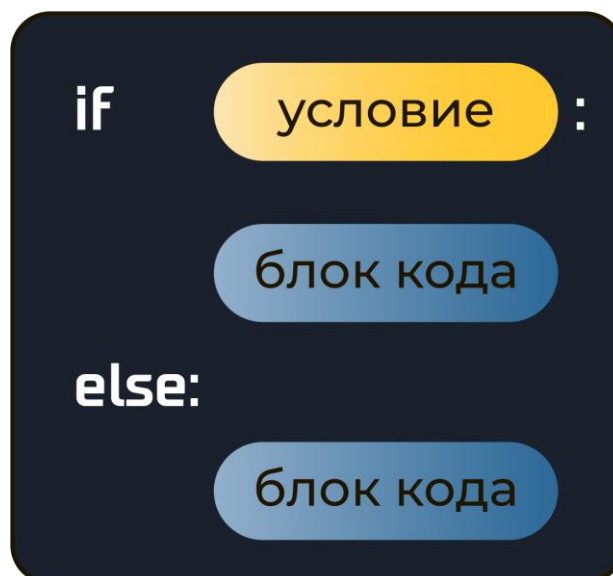
    print('Верно! Мы ботаем Python =)')

    print('Python - отличный язык!')

else:

    print('Не совсем так!')
```

В новой программе мы обрабатываем сразу два случая: если условие истинно (пользователь ввел «Python»), и если условие ложно (пользователь ввел что угодно, кроме «Python»).



Отступы

В некоторых языках программирования отступы — дело личного вкуса, и можно вообще обходиться без них. Однако в Python они неотъемлемая часть кода.

Именно отступ сообщает интерпретатору Python, где начинается и где заканчивается блок кода.

Отступ — небольшое смещение строки кода вправо. В начале такой строки находятся пробелы, и поэтому она на несколько символов отстоит от левого края.

Некоторым инструкциям в Python (например, инструкции `if`) именно блок кода сообщает, какие действия следует предпринять. После `if` блок кода информирует интерпретатор Python, как действовать, если условие истинно, и как — если оно ложно.

По соглашению PEP 8, для отступа блоков кода используются **4 пробела**. Если в среде Wing IDE нажать на клавишу Enter после `if`, она автоматически выставит **4 пробела**.

Операторы сравнения

Можно заметить, что в проверке условия мы использовали двойное равенство (`==`), вместо ожидаемого одиночного (`=`). Не стоит путать **оператор присваивания** (`=`) с **условным оператором** (`==`).

Оператор присваивания (`=`) присваивает переменным значения:

```
num = 1992
```

```
s = 'I love Python'
```

Для проверки двух элементов на равенство Python использует удвоенный знак равно (`==`). Вот так:

```
if answer == 'Python':
```

```
if name == 'Gvido':
```

```
if temperature == 40:
```

Путаница с операторами `==` и `=` является одной из самых распространенных ошибок в программировании. Эти символы используются не только в Python, и каждый день множество программистов используют их неправильно.

В Python существует 6 основных операторов сравнения.

Выражение	Описание
<code>if x > 7</code>	если x больше 7
<code>if x < 7</code>	если x меньше 7

Выражение	Описание
<code>if x >= 7</code>	если x больше либо равен 7
<code>if x <= 7</code>	если x меньше либо равен 7
<code>if x == 7</code>	если x равен 7
<code>if x != 7</code>	если x не равен 7

Рассмотрим пример:

```
num1 = int(input())
```

```
num2 = int(input())
```

```
if num1 < num2:
```

```
    print(num1, 'меньше чем', num2)
```

```
if num1 > num2:
```

```
    print(num1, 'больше чем', num2)
```

```
if num1 == num2:                                # используем двойное равенство
```

```
    print(num1, 'равно', num2)
```

```
if num1 != num2:
```

```
    print(num1, 'не равно', num2)
```

Цепочки сравнений

Операторы сравнения в Python можно объединять в цепочки (в отличие от большинства других языков программирования, где для этого нужно использовать логические связки), например, `a == b == c` или `1 <= x <= 10`. Следующий код проверяет, находится ли значение переменной `age` в диапазоне от 3 до 6:

```
age = int(input())
```

```
if 3 <= age <= 6:
```

```
    print('Вы ребёнок')
```

Код, проверяющий равенство трех переменных, может выглядеть так:

```
if a == b == c:
```

```
print('числа равны')

else:

    print('числа не равны')
```

Логические операторы

Как быть в ситуации, когда у нас есть несколько условий? В Python есть три логических оператора, которые позволяют создавать сложные условия:

- `and` — логическое умножение;
- `or` — логическое сложение;
- `not` — логическое отрицание.

Оператор and

Предположим, мы написали программу для учеников от двенадцати лет, которые учатся по крайней мере в 7 классе. Доступ к ней тем, кто младше, надо запретить. Следующий код решает поставленную задачу:

```
age = int(input('Сколько вам лет?: '))

grade = int(input('В каком классе вы учитесь?: '))

if age >= 12 and grade >= 7:

    print('Доступ разрешен.')

else:

    print('Доступ запрещен.')
```

Мы объединили два условия при помощи оператора `and`. Оно означает, что в этом ветвлении блок кода выполняется только при выполнении **обоих условий одновременно!**

Оператор `and` может объединять произвольное количество условий:

```
age = int(input('Сколько вам лет?: '))

grade = int(input('В каком классе вы учитесь?: '))

city = input('В каком городе вы живете?: ')

if age >= 12 and grade >= 7 and city == 'Москва':

    print('Доступ разрешен.')

else:
```

```
print('Доступ запрещен.')
```

Это таблица истинности для оператора `and`. В ней перечислены выражения, соединённые оператором `and`, показаны все возможные комбинации истинности и ложности и приведены результирующие значения выражений.

a	b	a and b
False	False	False
False	True	False
True	False	False
True	True	True

Как показывает таблица, чтобы значение выражения с оператором `and` было истинным, должны быть истинными **оба (все)** объединенные им условия.

Оператор or

Оператор `or` также применяется для объединения условий. Однако, в отличие от `and`, для выполнения блока кода достаточно выполнения **хотя бы одного из условий**.

```
city = input('В каком городе вы живете?: ')
```

```
if city == 'Москва' or city == 'Санкт-Петербург' or city ==  
'Екатеринбург':
```

```
    print('Доступ разрешен.')
```

```
else:
```

```
    print('Доступ запрещен.')
```

Доступ будет разрешен в случае, если хотя бы одно из условий выполнится.

Это таблица истинности для оператора `or`. В ней перечислены выражения, соединённые оператором `or`, показаны все возможные комбинации истинности и ложности и приведены результирующие значения выражений.

a	b	a or b
False	False	False
False	True	True

a	b	a or b
True	False	True
True	True	True

Для того, чтобы выражение `or` было истинным, требуется, чтобы **хотя бы одно** условие оператора `or` было истинным. При этом не имеет значения, истинным или ложным является второе выражение.

Логическое выражение `X and Y` истинно, если **оба** значения X и Y истинны.

Логическое выражение `X or Y` истинно, если **хотя бы одно** из значений X и Y истинно.

Мы можем использовать оба логических оператора одновременно:

```
age = int(input('Сколько вам лет?: '))
grade = int(input('В каком классе вы учитесь?: '))
city = input('В каком городе вы живете?: ')

if age >= 12 and grade >= 7 and (city == 'Москва' or city == 'Санкт-Петербург'):
    print('Доступ разрешен.')
else:
    print('Доступ запрещен.')
```

Такой код проверяет, что возраст учеников от двенадцати лет и учатся они по крайней мере в 7 классе и живут в Москве или Санкт-Петербурге.

Оператор not

Оператор `not` позволяет инвертировать (т.е. заменить на противоположный) результат логического выражения. Например, следующий код:

```
age = int(input('Сколько вам лет?: '))

if not (age < 12):
    print('Доступ разрешен.')
else:
    print('Доступ запрещен.')
```

полностью эквивалентен коду:

```
age = int(input('Сколько вам лет?: '))

if age >= 12:

    print('Доступ разрешен.')

else:

    print('Доступ запрещен.')
```

В первом примере мы поместили выражение `age < 12` в скобки для того, чтобы было чётко видно, что мы применяем оператор `not` к значению выражения `age < 12`, а не только к переменной `age`.

Таблица истинности для оператора `not`:

a	not a
False	True
True	False

Приоритеты логических операторов

Логические операторы, подобно арифметическим операторам (+, -, *, /), имеют приоритет выполнения. Приоритет выполнения следующий:

- в первую очередь выполняется логическое отрицание `not`;
- далее выполняется логическое умножение `and`;
- далее выполняется логическое сложение `or`.

Для **явного указания порядка** выполнения условных операторов **используют скобки**.

Примечания

Примечание 1. Частой ошибкой у начинающих программистов является путаница логических операторов `and` и `or`. Рассмотрим два условия:

```
if x > 1 and x < 100:

if x > 1 or x < 100:
```

Верным является только первое условие. В нём проверяется, что число x находится в диапазоне от 1 до 100, другими словами, $x \in (1; 100)$. Второе условие проверяет, что число x или больше 1, или меньше 100. Однако такому условию удовлетворяет любое число!

Примечание 2. Другую частую ошибку видим в следующем примере кода:

```
if age >= 7 and <= 9:
```

Запуск такого кода приведет к появлению ошибки во время выполнения программы. Необходимо явно записывать условия:

```
if age >= 7 and age <= 9:
```

Примечание 3. Не забывайте, что в Python есть удобный способ для проверки принадлежности диапазону. Например, следующий код:

```
if age >= 7 and age <= 9:
```

полностью эквивалентен коду:

```
if 7 <= age <= 9:
```

Последний код предпочтительнее.

Примечание 4. Оба оператора, `and` и `or`, вычисляются по **укороченной схеме**.

Вот как это работает с оператором `and`. Если условие слева от оператора `and` является ложным, то условие справа от него не проверяется, так как результат выражения будет гарантированно ложным и проверка оставшегося условия — пустая трата процессорного времени.

Аналогично работает оператор `or`. Если условие слева от оператора `or` истинное, то условие справа от него не проверяется. Действительно, результат будет гарантированно истинным и проверка оставшегося условия станет пустой тратой процессорного времени.

Вложенный оператор

Внутри условного оператора можно использовать любые инструкции языка Python, в том числе и условный оператор. Получаем вложенное ветвление: после одной развилки в ходе исполнения программы появляется другая развилка. При этом вложенные блоки имеют больший размер отступа (+4 пробела для каждого следующего уровня).

```
if условие1:
```

```
    блок кода
```

```
else:
```

```
    if условие2:
```

```
        блок кода
```

```
    else:
```

```
if условие3:
```

```
    блок кода
```

```
...
```

В предыдущем уроке мы разбирали задачу об определении координатной четверти точки. Программу можно переписать с использованием вложенного оператора:

```
x = int(input())
```

```
y = int(input())
```

```
if x > 0:
```

```
    if y > 0:
```

```
        print('Первая четверть')
```

```
    else:
```

```
        print('Четвертая четверть')
```

```
else:
```

```
    if y > 0:
```

```
        print('Вторая четверть')
```

```
    else:
```

```
        print('Третья четверть')
```

В данном случае уровень вложенности равен двум, так что программа одинаково хорошо читается как с помощью использования логического оператора `and`, так и с помощью вложенного оператора.

Рассмотрим программу, которая переводит стобалльную оценку в пятибалльную. Для её реализации нужно воспользоваться вложенным условным оператором:

```
grade = int(input('Введите вашу отметку по 100-балльной системе: '))
```

```
if grade >= 90:
```

```
    print(5)
```

```
else:
```

```
    if grade >= 80:
```

```
print(4)
```

```
else:
```

```
if grade >= 70:
```

```
print(3)
```

```
else:
```

```
if grade >= 60:
```

```
print(2)
```

```
else:
```

```
print(1)
```

В этом примере уровень вложенности настолько глубок, что код становится трудно понять.

Выбор из нескольких альтернатив – это обычное дело, здесь имеет смысл избегать глубокого вложения. Для этого в Python есть **каскадный условный оператор**.

Мы не могли написать 5 независимых `if`-ов, поскольку в таком случае было бы напечатано сразу несколько значений пятибалльной оценки.

Каскадный условный оператор

Если требуется проверить несколько условий, в языке Python используется **каскадный условный оператор**.

Синтаксис каскадного условного оператора имеет следующий вид:

```
if условие1:
```

```
    блок кода
```

```
elif условие2:
```

```
    блок кода
```

```
...
```

```
else:
```

```
    блок кода
```



При исполнении такого условного оператора сначала проверяется условие 1. Если оно является истинным, то выполняется блок кода, который следует сразу после него, вплоть до выражения `elif`. Остальная часть конструкции игнорируется. Однако если условие 1 является ложным, то программа перескакивает непосредственно к следующему выражению `elif` и проверяет условие 2. Если оно истинное, то выполняется блок кода, который следует сразу после него, вплоть до следующего выражения `elif`. И остальная часть условного оператора тогда игнорируется. Этот процесс продолжается до тех пор, пока не будет найдено условие, которое является истинным, либо пока больше не останется выражений `elif`. Если ни одно условие не является истинным, то выполняется блок кода после выражения `else`.

Приведенный ниже фрагмент кода является примером каскадного условного оператора `if-elif-else`. Этот фрагмент кода работает так же, как предыдущий код, использующий вложенный условный оператор.

```
grade = int(input('Введите вашу отметку: '))
```

```
if grade >= 90:
```

```
    print(5)

elif grade >= 80:

    print(4)

elif grade >= 70:

    print(3)

elif grade >= 60:

    print(2)

else:

    print(1)
```

Обратите внимание на выравнивание и выделение отступом, которые применены в инструкции `if-elif-else`: выражения `if`, `elif` и `else` выровнены и исполняемые по условию блоки выделены отступом.

Инструкция `if-elif-else` не является обязательной, потому что ее логика может быть запрограммирована вложенными инструкциями `if-else`. Однако длинная серия вложенных инструкций `if-else` имеет два характерных недостатка:

- программный код может стать сложным и трудным для восприятия;
- из-за необходимого выделения отступом продолжительная серия вложенных инструкций `if-else` может стать слишком длинной, чтобы целиком уместиться на экране монитора без горизонтальной прокрутки.

Логика инструкции `if-elif-else` обычно прослеживается легче, чем длинная серия вложенных инструкций `if-else`. И поскольку в инструкции `if-elif-else` все выражения выровнены, длина строк в данной инструкции, как правило, короче.

Запомни. Заключительный блок `else` в операторе `if-elif-else` является необязательным.